

Visualizing Specification Violations

Using Computer Vision to Analyze Requirements

Violations using Runtime Verification Tools

Elizabeth Sloan

Team Number 14

March 16, 2025

Mid Project Report for
HCI575: Computational Perception

Iowa State University
Spring 2025

ABSTRACT

Ensuring the safety and correctness of safety-critical systems requires rigorous verification techniques. However, the results of these verification processes are often difficult to interpret, limiting their accessibility beyond domain experts. This work presents a proof-of-concept approach for improving the understandability of runtime verification outputs through computer vision-based visualization. Using MATLAB for object detection and tracking, and R2U2 for runtime verification, we process video data to detect and highlight system violations in real-time. A simple test case involving a moving ball is used to illustrate the methodology: the ball's trajectory is tracked, predefined violation zones are monitored, and violations are visualized by dynamically changing the highlighting color around the ball and overlaying labels onto the original video. The system is evaluated based on detection accuracy, false positive/negative rates, and processing speed, with qualitative assessments on visualization effectiveness. This research demonstrates the feasibility of integrating computer vision techniques with formal verification frameworks, paving the way for future applications in more complex real-world scenarios where visualizing verification results can enhance system debugging, safety analysis, and formal methods accessibility.

1 Introduction

Safety-critical systems such as aircraft, banking infrastructure, nuclear reactors, and medical devices, must be rigorously verified to ensure they perform exactly as intended, with no unexpected behaviors. Failure in such systems can lead to catastrophic consequences, including financial loss, environmental disasters, or loss of human life. To prevent such failures, engineers can use formal methods, a set of mathematically rigorous techniques used to verify that both software and hardware adhere to strict safety and correctness requirements.

Formal methods provide strong guarantees about system behavior by producing mathematical proofs of correctness or counterexamples that highlight system failures. However, despite their power, formal methods are often perceived as inaccessible, primarily because they require specialized knowledge to interpret their results. Many formal verification tools produce complex outputs that are difficult to understand. This creates a significant accessibility gap, where only those deeply familiar with formal methods can effectively utilize the results, limiting their broader adoption in industry and multidisciplinary teams.

This work addresses this accessibility challenge by visualizing runtime verification results, making them more intuitive and interpretable. By leveraging computer vision techniques, we develop a system that maps formal verification outputs onto video data, allowing users to see violations and satisfactions directly in real-world scenarios. This proof-of-concept system integrates object detection, tracking, and real-time color-coded overlays to highlight contract violations in a visually intuitive manner. In doing so, we aim to bridge the gap between formal verification and practical debugging, enabling a broader audience beyond formal methods experts to leverage formal methods more effectively.

2 Related Work

Humans naturally excel at pattern recognition. We are also excellent at understanding and recognizing visual patterns, and we use visual patterns in recognizing and understanding complex ideas [16, 20]. The use of mathematical symbols and equations to introduce rigor to abstract concepts creates a difficulty in understanding the meaning of the concepts. Using visuals to is common practice in helping to understand the complexities that are solidified by mathematical rigor by allowing simplification of complex ideas. For example, when learning basic addition, we present someone with the equation $2+3=5$ as the abstract, mathematical rigor, but to help with learning the meaning behind the symbols, we may present the person with pictures of apples and ask them to count the apples to demonstrate the pattern of addition in an accessible, concrete manner [7]. This form of visual understanding of complex mathematical topics can also be seen in [3] where two children of elementary school ages create visual proofs of the concept of interleaving in programming. Understanding these concepts is deeply important for being able to use the concepts effectively and to be able to communicate the findings to others.

This type of pattern recognition has been acknowledged as a key aspect for understanding the mathematical rigor behind formal methods [4, 11, 22]. At Iowa State University’s Laboratory for Temporal Logic, two works have been published on the visualization of temporal logic. The WEST tool allows a user to input a Mission-Time Linear Temporal Logic equation, and it will display satisfying or dissatisfying traces (this is different from sat solving!) [5], and the ltl2timeline tool

creates timelines for Linear Temporal Logic [12]. There are also other works such as FRET which allows users to create formal requirements based on human-friendly text and shows timelines of the specifications to help with understandability of the generated requirements [8], and several online GUI's that also display visuals for formal methods [17, 15, 13, 10, 14, 6] demonstrating the desire in the field for more accessible visuals of formal languages.

One of the benefits of displaying specifications visually is in aiding requirements debugging. When creating the concept of operations (ConOps) for systems, engineer and design teams create hundreds or thousands of requirements for their systems. The size of these documents and requirements are an immediate barrier to understanding, and when some requirement contradicts another requirement, it takes substantial man hours to correct and is often infeasible to correct without automated help. Several works have addressed using visualizations for program debugging [9, 2, 1] all with the attempts to leverage human pattern recognition to aid in the understanding of complex, abstract, rigorous specification of large safety-critical systems.

Computer vision has been used for system monitoring to aid in work that is better suited to computers [19], as well as monitoring the safety of environments like traffic intersections [21]. However, due to the dichotomy of machine learning (AI) being provably unverifiable, and formal methods requiring rigorous verification, there are not many intersections between the two fields. One paper attempts to formalize a computer vision system to then be verified through formal methods [18], but that is not quite the same as mixing computer visualization with formal methods.

This work is one of the first to directly combine the fields of formal methods, specifically runtime verification, and computer vision to explore the usefulness of visualizations on a system in motion. The results from this study will add robustness to the field of formal methods understandability, as well as increasing the accessibility of formal methods by creating a new method for explaining requirements violations.

3 Experimental Platform

The experiments will use a video of a ball moving across a surface in different trajectories along with simple requirements such as “The ball will never cross over a certain line on the video frame”, which will be used to define regions of requirements violations. The video and requirements will be run through MATLAB for preprocessing, object detection, and object tracking, and the data will be saved and input into the runtime verification tool R2U2. R2U2 will produce a verdict of violation and that verdict along with the original video will be sent back into MATLAB to label and color code the ball as it moves across the screen.

3.1 Equipment

MATLAB serves as the primary development environment, with use of the Computer Vision Toolbox and Image Processing Toolbox for object detection, tracking, and color-based segmentation. Additionally, R2U2, a runtime verification framework, is employed to process extracted trajectory data and detect violations based on predefined constraints. The input data is collected using a phone camera, which records videos of a ball moving within a controlled environment. These videos serve as test cases for evaluating object tracking, violation detection, and real-time visualization. Depending on the project's complexity and implementation needs, Simulink may also be

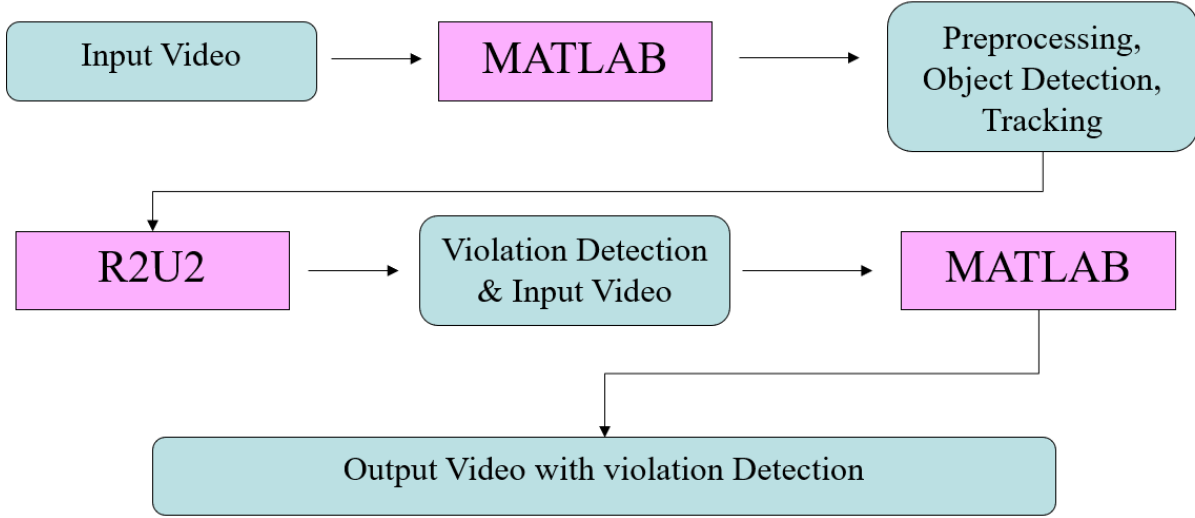


Figure 1: The data flow as described in paragraph 1 of section 3.

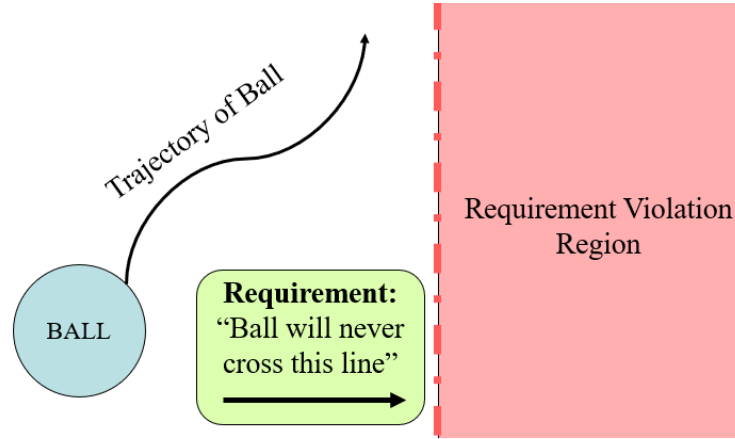


Figure 2: The ball will be rolled in a certian trajectory on the video. There will be a set of requirements and an area of interest in each frame.

incorporated to assist with model-based design and system-level visualization.

3.2 Data Structures

The input video is treated as a sequence of image matrices, where each frame is stored as a 2D array of pixel values, each frame will be treated as a timestep in R2U2. Object detection and tracking rely on lists and arrays to store positional data, including the coordinates of the ball, identified violation zones, and timestamped tracking information. Bounding boxes are used to define the spatial extent of detected objects. These bounding boxes also define regions of interest where violations are expected to occur, allowing for real-time comparison between the ball's trajectory and the predefined constraints.

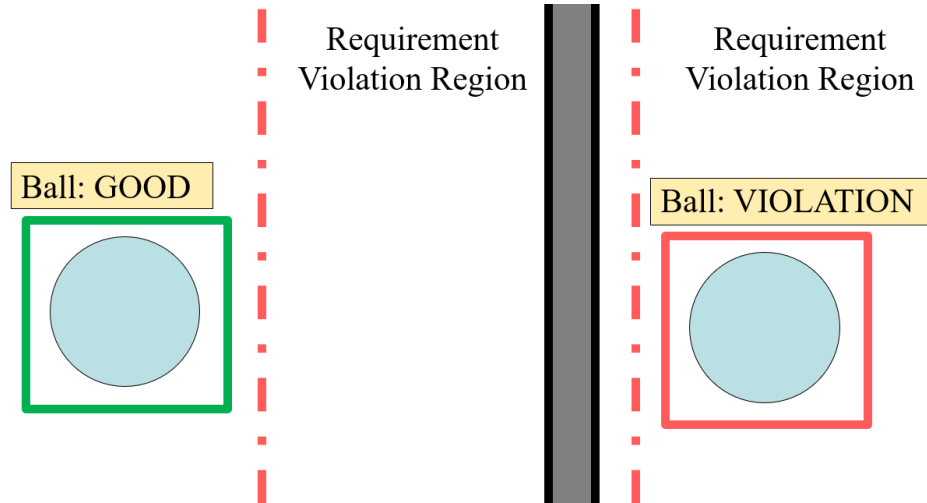


Figure 3: When the ball is outside the requirement violation region it will be highlighted in green and labeled with a positive note. When the ball enters the requirement violation region the highlight will change to red and the label will state what the requirement violation is.

3.3 Algorithms

Object detection will identify where the ball is within each video frame. This is accomplished through a combination of color-based filtering, blob detection, and the Hough Transform. Once the ball is detected, a tracking algorithm is employed to follow its movement across frames. The system will use Kalman Filters for predictive tracking and potentially Particle Filters for handling more complex motion patterns or partial occlusions.

Following object tracking, the violation detection module determines whether the ball enters restricted zones or satisfies specific conditions. This is achieved through geometric thresholding, which compares the ball's position to predefined spatial boundaries, and region of interest identification, which marks areas where violations occur. If a violation is detected, the system triggers a color coding and labeling mechanism, dynamically updating the visualization in real-time. The ball's outline changes to red when it enters a violation zone and reverts to green when it returns to a safe area. Additionally, a labeling algorithm overlays textual descriptions of detected violations directly onto the video.

3.4 Evaluation Methodology

3.4.1 Testing

The evaluation consists of both component testing and system-wide testing, ensuring that each module functions independently before being assessed as part of an integrated framework. Component testing verifies the accuracy of object detection, tracking, violation detection, and color coding in isolation. This involves testing detection algorithms under varying lighting conditions, evaluating tracking robustness in scenarios with occlusions, and confirming that color-coded overlays correctly represent violations.

Controlled experiments are conducted using pre-recorded videos of a ball moving in different

trajectories. These controlled datasets enable direct comparisons between expected and actual violation detections. Additionally, variations in violation criteria are applied to the same video to examine the flexibility of the system in handling different verification conditions. Edge case testing may also be performed, depending on time and extent of work, to ensure the system remains reliable under challenging conditions. This includes scenarios where the ball is temporarily hidden behind an object, experiences motion blur due to rapid movement, or is affected by changes in lighting.

3.4.2 Success Criteria

To quantify the system’s performance, a combination of quantitative and qualitative metrics is used. Detection accuracy is measured by comparing detected violations against a manually labeled ground truth dataset, allowing for precise evaluation of true positives, false positives, and false negatives. The system’s false positive and false negative rates are also analyzed to determine the reliability of violation detection. Processing speed is measured in terms of frame rate performance, assessing whether the system can operate efficiently enough to support real-time feedback.

In addition to quantitative analysis, qualitative metrics are considered to evaluate the interpretability of the visualized results. A subjective assessment will be conducted to determine how effectively the color-coded overlays and labels convey violation information. Future work may also include user studies, where participants interpret the visualizations to assess the system’s usability and effectiveness in enhancing formal verification understandability.

3.5 Potential Improvements

Based on evaluation results, several improvements may be considered to enhance system performance and usability. Object detection and tracking algorithms may be refined to improve accuracy under varying conditions, such as incorporating deep learning-based detection models for better generalization. Visualization refinements could also be introduced to improve the clarity of displayed information, such as creating temporal Boolean graphs that can be interacted with. Additionally, the framework should be extended to support more complex verification requirements, such as detecting violations involving multiple interacting objects or non-trivial temporal constraints, and more complex objects to be useful to verification as a field, instead of the abstract ball rolling across a screen.

References

- [1] Edward E Aftandilian, Sean Kelley, Connor Gramazio, Nathan Ricci, Sara L Su, and Samuel Z Guyer. Heapviz: interactive heap visualization for program understanding and debugging. In *Proceedings of the 5th international symposium on Software visualization*, pages 53–62, 2010.
- [2] Ron Baecker, Chris DiGiano, and Aaron Marcus. Software visualization for debugging. *Communications of the ACM*, 40(4):44–54, 1997.
- [3] Antonio Cerone. From stories to concurrency: How children can play with formal methods. In *Formal Methods–Fun for Everybody: First International Workshop, FMFun 2019, Bergen, Norway, December 2–3, 2019, Revised Selected Papers 1*, pages 191–207. Springer, 2021.
- [4] Nicolas Dulac, Thomas Viguier, Nancy Leveson, and M-A Storey. On the use of visualization in formal requirements specification. In *Proceedings IEEE Joint International Conference on Requirements Engineering*, pages 71–80. IEEE, 2002.
- [5] Jenna Elwing, Laura Gamboa-Guzman, Jeremy Sorkin, Chiara Travesset, Zili Wang, and Kristin Yvonne Rozier. Mission-time ltl (mtltl) formula validation via regular expressions. In *International Conference on Integrated Formal Methods*, pages 279–301. Springer, 2023.
- [6] Bernd Finkbeiner, Florian Kohn, and Malte Schledjewski. Leveraging static analysis: an ide for rtlola. In *International Symposium on Automated Technology for Verification and Analysis*, pages 251–262. Springer, 2023.
- [7] Karen C Fuson. Relating math words, visual images, and math symbols for understanding and competence. *International journal of disability, development and education*, 66(2):119–132, 2019.
- [8] Dimitra Giannakopoulou, Anastasia Mavridou, Julian Rhein, Thomas Pressburger, Johann Schumann, and Nija Shi. Formal requirements elicitation with fret. In *International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ-2020)*, number ARC-E-DAA-TN77785, 2020.
- [9] Frédéric Goualard and Frédéric Benhamou. A visualization tool for constraint program debugging. In *14th IEEE International Conference on Automated Software Engineering*, pages 110–117. IEEE, 1999.
- [10] Hannes Kallwies, Martin Leucker, Malte Schmitz, Albert Schulz, Daniel Thoma, and Alexander Weiss. Tessla—an ecosystem for runtime verification. In *International Conference on Runtime Verification*, pages 314–324. Springer, 2022.
- [11] Soon-Kyeong Kim and David Carrington. Visualization of formal specifications. In *Proceedings Sixth Asia Pacific Software Engineering Conference (ASPEC’99)(Cat. No. PR00509)*, pages 102–109. IEEE, 1999.

- [12] Runming Li, Keerthana Gurushankar, Marijn JH Heule, and Kristin Yvonne Rozier. What’s in a name? linear temporal logic literally represents time lines. In *2023 IEEE Working Conference on Software Visualization (VISSOFT)*, pages 73–83. IEEE, 2023.
- [13] Leonardo Lima, Andrei Herasimau, Martin Raszyk, Dmitriy Traytel, and Simon Yuan. Explainable online monitoring of metric temporal logic. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 473–491. Springer, 2023.
- [14] Leonardo Lima, Jonathan Julián Huerta y Munive, and Dmitriy Traytel. Whymon: A runtime monitoring tool with explanations as verdicts. In *International Symposium on Automated Technology for Verification and Analysis*, pages 76–90. Springer, 2024.
- [15] Nuno Macedo, Alcino Cunha, José Pereira, Renato Carvalho, Ricardo Silva, Ana CR Paiva, Miguel S Ramalho, and Daniel Silva. Sharing and learning alloy on the web. *arXiv preprint arXiv:1907.02275*, 2019.
- [16] Frank M Marchak, William S Cleveland, Bernice E Rogowitz, and Christopher D Wickens. The psychology of visualization. In *IEEE Visualization: Proceedings of the 4 th conference on Visualization’93: San Jose, California*, volume 25, pages 351–354, 1993.
- [17] Tim Nelson, Ben Greenman, Siddhartha Prasad, Tristan Dyer, Ethan Bove, Qianfan Chen, Charles Cutting, Thomas Del Vecchio, Sidney LeVine, Julianne Rudner, et al. Forge: A tool and language for teaching formal methods. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):613–641, 2024.
- [18] Oleksandr Semeniuta and Petter Falkman. Discrete event dataflow as a formal approach to specification of industrial vision systems. In *2015 IEEE International Conference on Automation Science and Engineering (CASE)*, pages 849–854. IEEE, 2015.
- [19] Albert TP So and WL Chan. A computer-vision based power plant monitoring system. In *1991 International Conference on Advances in Power System Control, Operation and Management, APSCOM-91.*, pages 335–340. IET, 1991.
- [20] David H Uttal and Katherine O’ Doherty. Comprehending and learning from ‘visualizations’: A developmental perspective. In *Visualization: Theory and practice in science education*, pages 53–72. Springer, 2008.
- [21] Harini Veeraraghavan, Osama Masoud, and Nikolaos P Papanikolopoulos. Computer vision algorithms for intersection monitoring. *IEEE Transactions on Intelligent Transportation Systems*, 4(2):78–89, 2003.
- [22] Thomas Viguier. *Evaluating visualization in formal requirements specification: an experiment with human subjects*. PhD thesis, Massachusetts Institute of Technology, 2003.